

Como você entendeu o problema e por onde decidiu começar?

Eu achei o documento enviado pela Sara bem claro já, não me restaram dúvidas. Pensando nos objetivos da empresa, a necessidade de um projeto como esse é evidente.

Eu resolvi começar pelo pipeline, para poder desenvolver o frontend e o backend já enviando e recebendo dados reais. Mesmo assim, depois da primeira versão do ETL, comecei a desenvolver o front e back juntos, e, dependendo da dificuldade, optava pela resolução na camada mais vantajosa.

Quais foram suas principais decisões e estratégias, o que você descartou?

O objetivo principal era permitir um uso rápido, leve e consistente do pipeline, que permitisse fugir de uma estratégia de download de arquivos. Naturalmente, Python se mostrou como candidato, mas, pensando no futuro do projeto, principalmente em permitir um controle via frontend e backend do ETL, optei pela linguagem JavaScript, que não gerou perda no tempo de execução e consumo de RAM. Nos dados da CVM, consegui entender a estrutura de links e, pelo fetch(), receber somente o arquivo desejado. Já nas informações de negociação na ANBIMA, não consegui encontrar um tipo de link tão fácil quanto o da CVM, e a opção de download envolvia telas de login e verificação de e-mail. Nesse caso, o web scraping com Selenium foi a opção escolhida.

Sobre o desenho do banco de dados, optei pela divisão em tabelas `fi_cadastro`, que recebia seus dados principalmente do `cad_fi.csv` do CVM; `fi_carteira`, dos arquivos do `cda_fi_....zip`; `fi_informe`, dos dados da `inf_diario_....zip/csv`; `fi_rentabilidades`, das informações de `lamina_....zip`; e, finalmente, `negociacao_titulos_publicos`, do web scraping do site da ANBIMA.

Esse desenho se vincula diretamente com a separação por assuntos na base e também com uma arquitetura que tente otimizar a quantidade de memória usada, bem como facilitar o uso de estratégias de bloqueio de duplicidades e identificação de inconsistências. Durante os testes, verifiquei que poucos fundos tinham informações em todas as tabelas: menos de 1.000. Assim, além do `cad_fi.csv`, comecei a utilizar as outras planilhas para identificação de CNPJs de fundos.

Inicialmente, havia planejado o frontend para receber as informações da base durante o carregamento da página, para diminuir o número de requisições ao servidor, utilizando ao máximo a capacidade computacional do usuário. Porém, pela quantidade de memória utilizada e lentidão e instabilidade do site, rapidamente percebi que essa estratégia não teria futuro conforme a base aumentasse.

Assim, optei por um equilíbrio maior entre o uso computacional do servidor e do site, aumentando o número de requisições. Em outras palavras, mudei o escopo do projeto de uma lógica de disponibilização para uma lógica de pesquisa. Pelas perspectivas futuras do projeto, escolhi desenvolver ele como uma Single Page Application, ao invés de usar o roteamento por URL. Essa decisão trouxe mais fluidez para a experiência do usuário, e permitiu que as consultas fossem mais rápidas e leves do que se envolvessem constantes carregamentos de páginas.

O MVP do projeto seriam duas telas, uma para consulta de informações dos fundos (optando pelas colunas que menos tinham valores NULOS e permitiam a identificação

correta do fundo e sua situação) e outra para informações das negociações públicas. O objetivo era uma estrutura de arquivos e de design que permitisse desenvolver mais telas conforme os usos e necessidades da empresa, inclusive mirando automações internas.

Assim, como demonstração, uma terceira tela foi adicionada, disponibilizando visões gerais de toda a base; como não queria limitar as possibilidades de uso e variedade pela minha criatividade, utilizei nessa tela uma ideia de chatbot, permitindo que até um usuário pouco experiente tenha acesso amplo a pesquisas na base de dados. Com o uso do projeto, a ideia era manter o chatbot na tela, porém adicionar opções de geração de gráficos e relatórios com base nos assuntos e temas mais pesquisados.

O stack utilizado foi um próprio. Como desenvolvi ele com objetivos de SPA, rapidez e leveza, ele correspondeu bem com as necessidades e objetivos do desenvolvimento do projeto. Para facilitar ainda mais o uso, ele foi feito responsivo para mobiles. Desenvolvi o frontend e o backend concomitantemente, adaptando-os mutuamente aos desafios encontrados.

No backend, a ideia era selecionar as informações da base da forma mais leve possível. Assim, optei ao máximo pelo uso de queries SQL. Seguindo a lógica do frontend, inicialmente o backend enviava todas as informações em uma requisição inicial, o que se demonstrou bem demorado e pouco vantajoso em termos de uso de memória. Migrando para a lógica de pesquisa, organizei ele em funções de pesquisa globais e diretas. Como as informações são muitas, visando evitar a lentidão da lógica inicial, limitei os resultados de pesquisa em 500 itens por vez. Para a implementação do chatbot, utilizei uma IA interna do Ollama em duas etapas: formulação de query SQL útil para o pedido e formulação final da resposta. Os prompts foram desenvolvidos pensando nos casos de usos iniciais da tela, mas deveriam ser refinados caso novos usos aparecessem.

Onde você usou IA e onde não usou? Como validou o que ela produziu?

Usei IA principalmente no backend para refinamento da criação das queries SQL. O método foi criar eu mesmo as queries, validar o uso no SQL, enviar para a IA buscando aumentar a eficiência do código e então a validação novamente no SQL. Também usei ela para tirar dúvidas sobre funções e possibilidades na linguagem escolhida, bem como aumentar a eficiência do código no geral, avaliando se as melhoras eram positivas ou não, pensando também no futuro de sua manutenção.

Notadamente, usei IA como ferramenta principal para a tela de estatísticas.

Pelo stack escolhido, a IA não teve usabilidade durante o desenvolvimento do frontend.